

PL/SQL QUERIES

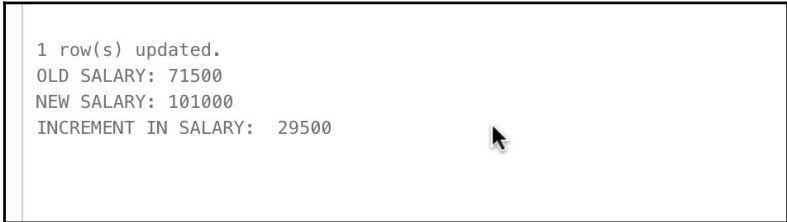
1. Creating Trigger to display old Salary and new updated Salary after every Updation.

Query:

```
CREATE OR REPLACE TRIGGER Update_Salary
BEFORE UPDATE
ON GRADE_PAY
FOR EACH ROW
DECLARE
SAL_DIFF NUMBER;
BEGIN
SAL_DIFF:=(:NEW.BASIC_PAY+:NEW.HRA+:NEW.DA)-(:OLD.BASIC_PAY+:OLD.HRA+:OLD.DA);
DBMS_OUTPUT.PUT_LINE('OLD SALARY: '||:OLD.BASIC_PAY+:OLD.HRA+:OLD.DA);
DBMS_OUTPUT.PUT_LINE('NEW SALARY: '||:NEW.BASIC_PAY+:NEW.HRA+:NEW.DA);
DBMS_OUTPUT.PUT_LINE('INCREMENT IN SALARY: '||:SAL_DIFF);
END;

UPDATE GRADE_PAY SET BASIC_PAY=80000 , HRA=8000 , DA=13000 WHERE GRADE_ID=6;
```

Output:



```
1 row(s) updated.
OLD SALARY: 71500
NEW SALARY: 101000
INCREMENT IN SALARY: 29500
```

2. Creating Cursor to display details of all Professors.

Query:

```
DECLARE
CURSOR C1 IS SELECT * FROM FACULTY WHERE GRADE_ID=1;
BEGIN
FOR REC IN C1
LOOP
DBMS_OUTPUT.PUT_LINE('FACULTY ID: '||REC.FAC_ID||' FACULTY NAME: '||REC.FAC_NAME||' DOB: '||REC.DOB||' EXPERIENCE: '||REC.EXPERIENCE||' CITY: '||REC.ADDRESS);
DBMS_OUTPUT.PUT_LINE(' ');
END LOOP;
END;
/
```

Output:

```

Statement processed.
FACULTY ID: 2  FACULTY NAME: SEEMA BAWA  DOB: 29-DEC-75  EXPERIENCE: 14  CITY: JALANDHAR
FACULTY ID: 4  FACULTY NAME: MANINDER SINGH  DOB: 29-APR-74  EXPERIENCE: 17  CITY: PATIALA
FACULTY ID: 17  FACULTY NAME: RS KALER  DOB: 19-JUN-70  EXPERIENCE: 17  CITY: LUDHIANA
FACULTY ID: 23  FACULTY NAME: AJAY BATISH  DOB: 02-JUN-70  EXPERIENCE: 20  CITY: LUDHIANA
FACULTY ID: 29  FACULTY NAME: RAJEEV MEHTA  DOB: 12-MAY-76  EXPERIENCE: 22  CITY: LUDHIANA
FACULTY ID: 35  FACULTY NAME: PRAKASH GOPLAN  DOB: 12-DEC-65  EXPERIENCE: 26  CITY: MUMBAI
FACULTY ID: 11  FACULTY NAME: ALPANA AGGARWAL  DOB: 29-NOV-70  EXPERIENCE: 17  CITY: PATIALA
FACULTY ID: 1  FACULTY NAME: RK SHARMA  DOB: 29-DEC-70  EXPERIENCE: 17  CITY: PATIALA

```

3. Using implicit Cursor to display No of Records deleted when deleting subjects of semester 3 .

Query:

```

DECLARE
ROWS_DELETED NUMBER;
BEGIN
DELETE FROM COURSE_SUBJECT WHERE SEMESTER=3;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' RECORDS DELETED');
END;
/

```

Output:

```

Statement processed.
14 RECORDS DELETED

```

4. Creating Trigger for displaying Old Experience and new Experience whenever we update the experience.

Query:

```

CREATE OR REPLACE TRIGGER UPDATE_FAC
BEFORE UPDATE
ON FACULTY
FOR EACH ROW
BEGIN
IF (:NEW.EXPERIENCE)<(:OLD.EXPERIENCE)) THEN
DBMS_OUTPUT.PUT_LINE('New Experience cannot be less than Old Experience');
:NEW.EXPERIENCE:=:OLD.EXPERIENCE;
ELSE
DBMS_OUTPUT.PUT_LINE('Old Experience: ' || :OLD.EXPERIENCE);
DBMS_OUTPUT.PUT_LINE('New Experience: ' || :NEW.EXPERIENCE);
END IF;
END;
/

```

Output:

Case 1:

```

BEGIN
UPDATE FACULTY SET EXPERIENCE=15 WHERE FAC_ID=32;
IF SQL%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('Record not Found');
END IF;
END;

```

/

```
Statement processed.  
Old Experience: 13  
New Experience: 15
```

Case 2:

```
BEGIN  
UPDATE FACULTY SET EXPERIENCE=15 WHERE FAC_ID=50;  
IF SQL%NOTFOUND THEN  
DBMS_OUTPUT.PUT_LINE('Record not Found');  
END IF;  
END;  
/
```

```
Statement processed.  
New Experience can not be less than Old Experience
```

Case 3:

```
BEGIN  
UPDATE FACULTY SET EXPERIENCE=2 WHERE FAC_ID=13;  
IF SQL%NOTFOUND THEN  
DBMS_OUTPUT.PUT_LINE('Record not Found');  
END IF;  
END;  
/
```

```
Statement processed.  
Record not Found
```

5. Showing details of departments using cursor when given Dept_Id exists and showing error using exception handling when given Dept_Id doesn't exist .

Case 1:

Query:

```
DECLARE  
CURSOR C(D_ID NUMBER) IS SELECT * FROM DEPT WHERE DEPT_ID=D_ID;  
REC DEPT%ROWTYPE;  
DNO NUMBER;  
DEPT_NOFOUND EXCEPTION;  
BEGIN  
DNO:=5;  
OPEN C(DNO);  
LOOP
```

```

FETCH C INTO REC;
EXIT WHEN C%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('DEPARTMENT ID: ' || REC.DEPT_ID);
DBMS_OUTPUT.PUT_LINE('DEPARTMENT NAME: ' || REC.DEPT_NAME);
DBMS_OUTPUT.PUT_LINE('LOCATION: ' || REC.LOCATION);
END LOOP;
IF C%ROWCOUNT=0 THEN
RAISE DEPT_NOFOUND;
END IF;
CLOSE C;
EXCEPTION
WHEN DEPT_NOFOUND THEN
DBMS_OUTPUT.PUT_LINE('DEPARTMENT WITH DEPT_ID: ' || DNO || ' DOESNOT EXISTS');
END;
/

```

Output:

```

Statement processed.
DEPARTMENT ID: 5
DEPARTMENT NAME: Chemical Engineering
LOCATION: E Block

```

Case 2:

```

DECLARE
CURSOR C(D_ID NUMBER) IS SELECT * FROM DEPT WHERE DEPT_ID=D_ID;
REC DEPT%ROWTYPE;
DNO NUMBER;
DEPT_NOFOUND EXCEPTION;
BEGIN
DNO:=12;
OPEN C(DNO);
LOOP
FETCH C INTO REC;
EXIT WHEN C%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('DEPARTMENT ID: ' || REC.DEPT_ID);
DBMS_OUTPUT.PUT_LINE('DEPARTMENT NAME: ' || REC.DEPT_NAME);
DBMS_OUTPUT.PUT_LINE('LOCATION: ' || REC.LOCATION);
END LOOP;
IF C%ROWCOUNT=0 THEN
RAISE DEPT_NOFOUND;
END IF;
CLOSE C;
EXCEPTION
WHEN DEPT_NOFOUND THEN
DBMS_OUTPUT.PUT_LINE('DEPARTMENT WITH DEPT_ID: ' || DNO || ' DOESNOT EXISTS');
END;
/

```

Output:

```

Statement processed.
DEPARTMENT WITH DEPT_ID: 12 DOESNOT EXISTS

```

6. Showing use of a Function in PL_SQL by printing total no of employees working in the University.

Query:

```
CREATE OR REPLACE FUNCTION FAC_COUNT
RETURN NUMBER
AS
COUNT_F NUMBER;
BEGIN
SELECT COUNT(*) INTO COUNT_F FROM FACULTY;
RETURN COUNT_F;
END;

DECLARE
COUNT_F NUMBER;
BEGIN
COUNT_F:=FAC_COUNT();
DBMS_OUTPUT.PUT_LINE('TOTAL NUMBER OF EMPLOYEES CURRENTLY WORKING IN UNIVERSITY = ' ||
COUNT_F);
END;
```

Output:

```
Statement processed.
TOTAL NUMBER OF EMPLOYEES CURRENTLY WORKING IN UNIVERSITY = 42
```